

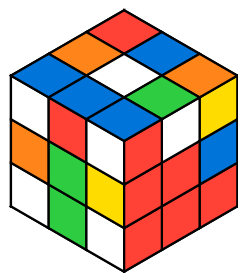
magic-cubes

v0.0.1 MIT

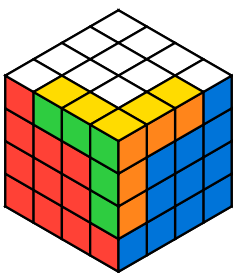
Represent Rubik's cubes of arbitrary size and apply algorithms to them

Rodrigo Alcántara

 @RodAlc24



U' L' U' F' R2 B' R
F U B2 U B' L U' F
U R F'



F U2 L F L' B L U
B' R' L' U R' D' F'
B R2

Table of Contents

About	2	Index	14
Examples	3		
Guide	6		
III.1 Creating cubes	6		
III.2 Applying algorithms	7		
III.3 Rendering cubes	7		
III.4 Cube notation	7		
III.4.1 1-layer moves	7		
III.4.2 Central moves	8		
III.4.3 Wide moves	9		
III.4.4 Cube rotations	9		
III.4.5 Modifiers	9		
Reference	10		
IV.1 Custom types	10		
IV.1.1 <code>cube</code>	10		
IV.1.2 <code>cube-colors</code>	10		
IV.1.3 <code>cube-state</code>	10		
IV.2 Functions	11		

Part I

About

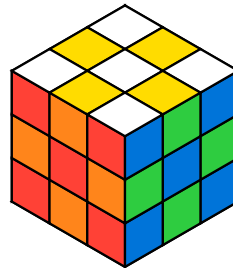
In 1974, Ernő Rubik invented a mechanical puzzle and called it the *Magic Cube*. Years later, in 1980, the puzzle was renamed the *Rubik's Cube*, the name by which it is now known. Today, it is considered to be the world's bestselling puzzle game.

[magic-cubes](#) is a package built on top of [CeTZ](#) that allows you to create, manipulate and render Rubik's cubes of any size.

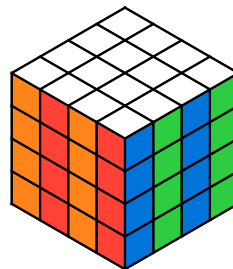
Part II

Examples

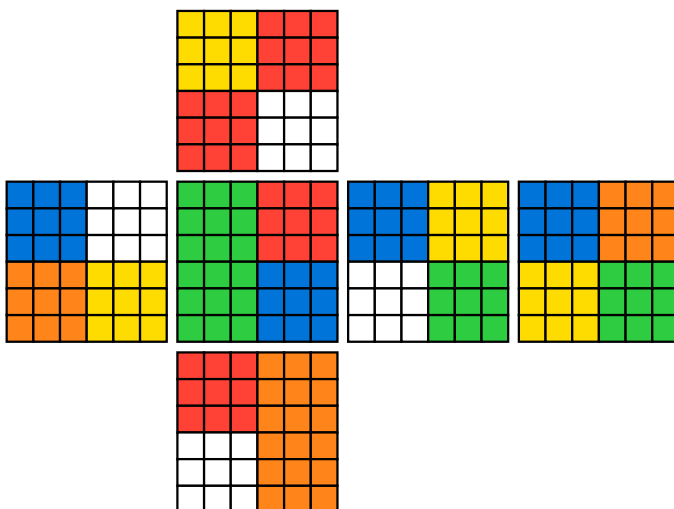
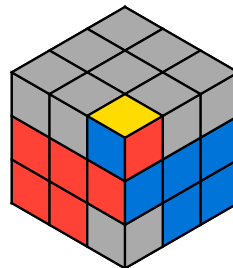
```
#draw_cube(
  apply(
    cube(),
    "F2 B2 R2 L2 U2 D2"
  )
)
```



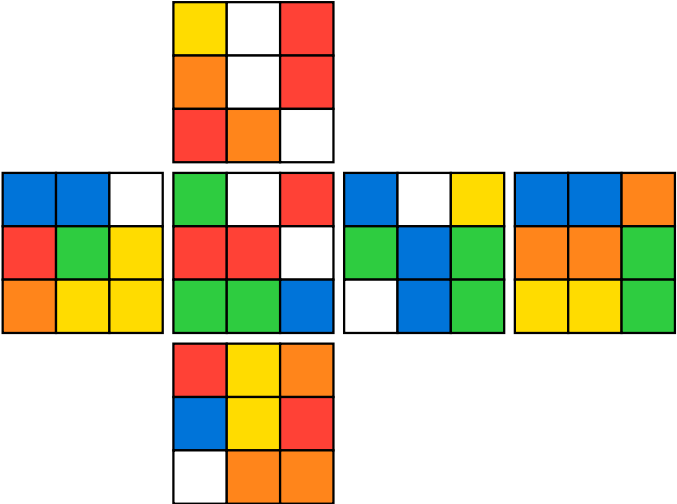
```
#draw_cube(
  apply(
    cube(size: 4),
    "R2 3R2 F2 3F2 R2 3R2"
  )
)
```



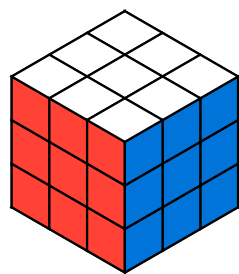
```
#draw_cube(
  apply(
    inverted: true,
    f2l-cube,
    "R2 U R2 U R2 U2 R2"
  )
)
```



II Examples



II Examples



,

Part III

Guide

III.1 Creating cubes

A cube is represented as a dictionary, and can be easily created with the `#cube` function. It should not be created or modified manually.

The size of the cube can be specified with the `<size>` argument (default is 3). The face colors can be changed with the `<colors>` argument, specifying the faces to change. The default colors are:

```
(  
  f: red,  
  r: blue,  
  u: white,  
  b: orange,  
  l: green,  
  d: yellow  
)
```

This colors are the Typst [predefined colors](#)¹.

It is also possible to specify a face manually, in that case the corresponding value in `<colors>` will be ignored. When specifying the colors manually, all the pieces in the face must be determined.

The order of the pieces in a face is the following:

			0	1	2			
			3	4	5			
			6	7	8			
0	1	2	0	1	2	0	1	2
3	4	5	3	4	5	3	4	5
6	7	8	6	7	8	6	7	8
			0	1	2			
			3	4	5			
			6	7	8			

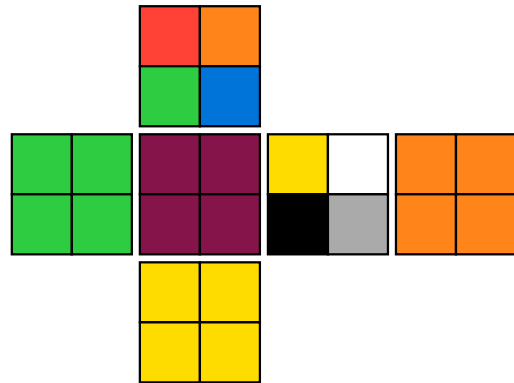
Each face is represented with the default color previously explained, except the up face that is in black.

Note that this way of creating cubes may result in an impossible cube state (you can even use more than six colors!). This can be useful, for example, to add a gray color to the non relevant pieces.

¹<https://typst.app/docs/reference/visualize/color/#predefined-colors>

This order is also used in other size cubes, for example, if we want to create a 2x2x2 cube with a custom color in front face and custom up and right faces we can do:

```
#draw_flat(
  length: 15pt,
  cube(
    size: 2,
    colors: (f: maroon),
    state: (
      u: (
        red,
        orange,
        green,
        blue,
      ),
      r: (
        yellow,
        white,
        black,
        gray,
      ),
    ),
  )
)
```



There are also some useful predefined cubes:

```
#draw_cube(f2l-cube, length: 10pt)
```



```
#draw_cube(oll-cube, length: 10pt)
```



III.2 Applying algorithms

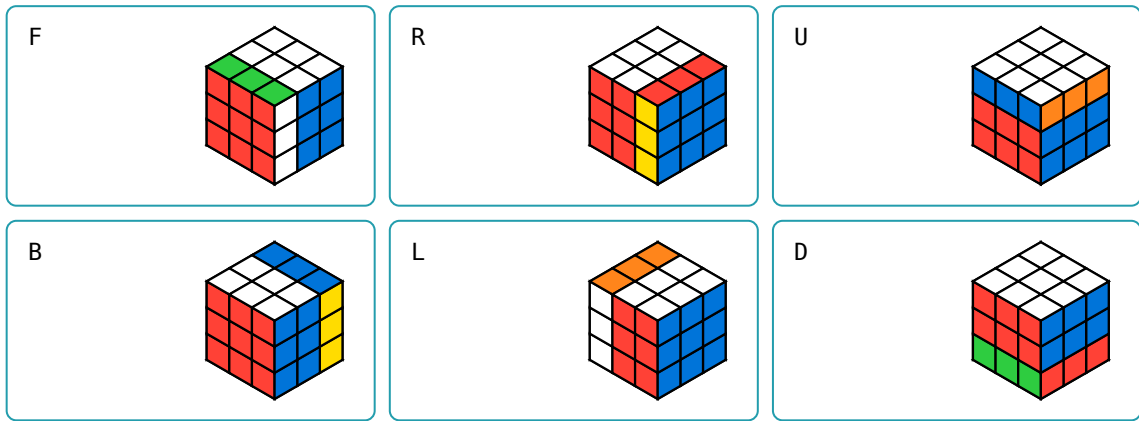
One of the main features of [magic-cubes](#) is the possibility to apply algorithms to the cubes. Algorithms are written following the standard notation, it is fully documented on [Section III.4](#)

III.3 Rendering cubes

III.4 Cube notation

III.4.1 1-layer moves

The basic moves are represented with one uppercase letter. There are six moves, one for each face of the cube: **F** (front), **R** (right), **U** (up), **B** (back), **L** (left) and **D** (down). All of them are a single clockwise rotation, double and anticlockwise rotations are explained later on [Section III.4.5](#).

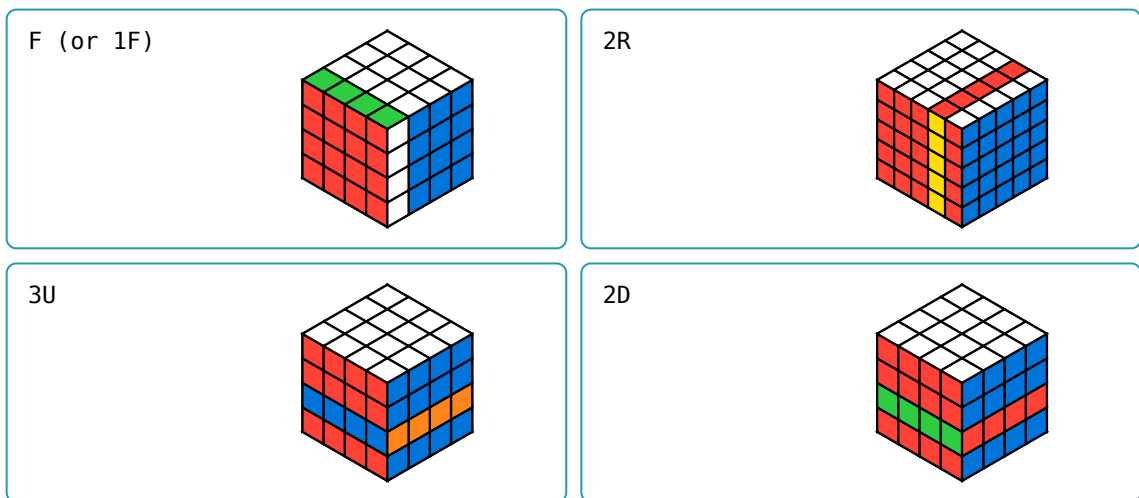


In big cubes it is possible to move an inner layer. That is done by preceding the move letter with the depth of the layer, with the outermost layer having a depth of 1.

It is not necessary to specify the depth for the moves of the outermost layer. For example, 1U is equivalent to U in all cubes.

All the rotations are done in clockwise, for double turns and anticlockwise turns see [Section III.4.5](#).

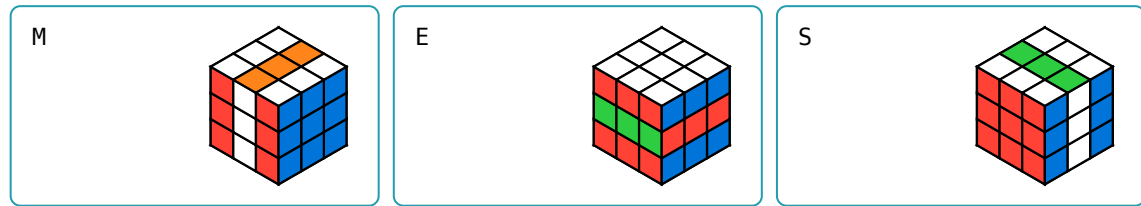
The maximum depth is size - 1, if you want to turn the opposite layer use the corresponding notation, i.e., in a 4x4x4 cube a 4F rotation is not allowed, the correct notation is B'.



III.4.2 Central moves

The central layers have a special notation:

- **M**: middle, between L and R, turn direction as L.
- **E**: equator, between U and D, turn direction as D.
- **S**: standing, between F and B, turn direction as F.



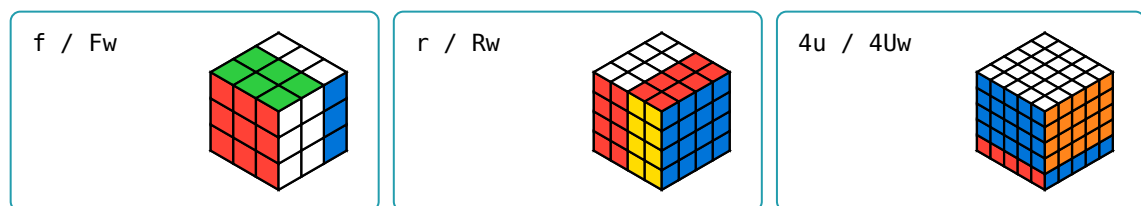
Central moves are only available on cubes with odd size.

III.4.3 Wide moves

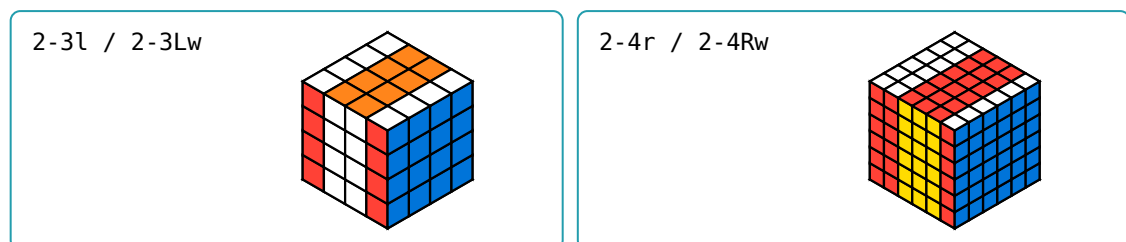
It is also possible to rotate more than one layer at the same time. This is specially useful for big cubes, but it is also used sometimes on a 3x3x3.

There are two equivalent notations: using lowercase letters or appending a **w** after the corresponding uppercase letter. By default, all layers between the outermost and the center (in case the cube has odd size) are rotated, but this can be specified with a number before the move. For example, a **3f** (or **3Fw**) will rotate the first 3 front layers.

Just like it happened with the depth parameter before, the maximum number of layers that can be moved with a wide move is equal to size - 1. For rotating the full cube it exists a specific notation that will be explained next.



It is also possible to rotate more than one inner layers, for that, you should write before the move the first and the last layer to rotate.



III.4.4 Cube rotations

All of them represent a single **clockwise** turn in the respective face.

III.4.5 Modifiers

Adding an apostrophe (') or a "2" after the move will cause an anticlockwise turn or a double one (180°) respectively.

Part IV

Reference

IV.1 Custom types

IV.1.1 **cube**

cube is the main type in this package, as it represents the state of a cube. However, you should not create or modify instances manually, as functions such as `#draw_cube` require the cube to be in a consistent state. Instead, use `#cube` to create instances and `parse` to manipulate them.

```
(  
  {size} number  
  {f} array of color  
  {r} array of color  
  {u} array of color  
  {b} array of color  
  {l} array of color  
  {d} array of color  
)
```

The length of the dictionaries must be equal to the square of the size.

IV.1.2 **cube-colors**

This type is used when creating a cube to specify the color of each face.

```
(  
  {f} color  
  {r} color  
  {u} color  
  {b} color  
  {l} color  
  {d} color  
)
```

Not all keys need to be present for a valid **cube-colors** element.

IV.1.3 **cube-state**

This type is used when creating a cube to specify the pieces of each face.

```
(
  {f} array of color
  {r} array of color
  {u} array of color
  {b} array of color
  {l} array of color
  {d} array of color
)
```

Not all keys need to be present for a valid `cube-state` element.

IV.2 Functions

```
#apply          #draw_cube
#cube           #draw_flat
```

#apply(`{cube}`, `{alg}`, `{inverted}`): `false` → `cube-t`

Returns a new cube after applying the algorithm.

Argument — `{cube}` `cube-t`
 The cube to apply the algorithm.

Argument — `{alg}` `str`
 The algorithm to apply.

Argument — `{inverted}`: `false` `bool`
 Whether it should be applied in inverse order.

#cube(`{size}`): `3`, `{colors}`: `auto`, `{state}`: `auto`) → `cube`

This function creates a `cube` given the size, colors, and/or state. This is the correct way to create `cube` instances.

The `{state}` argument lets you specify all the pieces (colors) of a face. Note that if a face is specified with this argument, it must be fully specified, and the corresponding color given in `{colors}` is not used.

Argument — `{size}`: `3` `int`
 The size of the cube. The default value is 3, i.e., a standard cube.

Argument

`{colors}: auto``cube-colors` | `auto`

Colors of each face of the cube.

It must be a `cube-colors`, although it does not need to contain all the keys. In that case, the default color value will be used.

The default values for each face are:

```
(
  f: red,
  r: blue,
  u: white,
  b: orange,
  l: green,
  d: yellow
)
```

Argument

`{state}: auto``cube-state` | `auto`

The state of the cube, the default is the solved state. See [Section III.1](#) for more details and examples.

```
#draw_cube(
  {cube},
  {x}: 35.264deg,
  {y}: 45deg,
  {z}: 0deg,
  {length}: 20pt
)
```

Draws a cube

Argument

`{cube}``cube-t`

the cube to draw.

Argument

`{x}: 35.264deg``angle`

the x angle.

Argument

`{y}: 45deg``angle`

the y angle.

Argument

`{z}: 0deg``angle`

the z angle.

#draw_flat(**{cube}**, **{length}**: **20pt**)

Draws a flat cube.

Argument

{cube}

cube - t

The cube to draw.

Part V

Index

A

`#apply` 11

C

`#cube` 6, 10, 11

`cube` 10

`cube-colors` 10

`cube-state` 11

D

`#draw_cube` 10, 11, 12

`#draw_flat` 11, 13